

Moazzamuddin Report

by Moazzamuddin Ahmed

Submission date: 28-Apr-2026 03:18PM (UTC+0500)

Submission ID: 2946501187

File name: CONNECT_4_OOP_Final_Project_Report.docx (663.12K)

Word count: 2270

Character count: 12627



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

PROJECT REPORT

CL1004
OBJECT ORIENTED PROGRAMMING

Prepared BY:

MOAZAMUDDIN 25K-0771

M. AHMED ASHFAQ

25K-0781



1: INTRODUCTION & ABSTRACT

Connect 4 is a game that two players play. They take turns dropping pieces into a grid that has seven columns and six rows. The goal of Connect 4 is to get four pieces in a row either across, up and down or diagonally before the other player does.

The first version of this project is a Connect 4 game written in C using the Raylib library. Two players can play this game. It has some big problems. The code is not well organized. It is hard to understand. For example the code does not separate the game logic from the visualization part. This makes it hard to change or fix things.

We want to make a version of Connect 4 that works better. We will use object-oriented programming to make the game more efficient. This means we will use things, like Encapsulation, Abstraction, Inheritance and Polymorphism to create an algorithm. The new game will have small parts that each do a specific job. This will make it easy to change the game or add features. We can even add a computer player that can think for itself, which is called Artificial Intelligence. Connect 4 will be a lot more fun when it is done.

2: TECHNOLOGY STACK

I have chosen to use the C++ programming language. The reason behind choosing C++ programming language is as it contains classes and objects which're really useful and easy to use. The C++ programming language also has a template library, which's a big plus. The C++ programming language has pointers, which make it easier for me to manage memory. This means I can directly reference the data or its address to avoid bugs that were in the C code.

I am using Raylib library for this project to give GUI based touch. Raylib lets me make windows draw shapes and work with input devices. Raylib also has several fonts which figure out how frames per second my game is running. In my project Raylib is used in a class called Renderer, which's a good way to organize things. No other classes in my project use the Raylib library directly.

The software application used is Visual Studio Code. GCC Compiler is used for compiling the program. The g++ linker creates the file on its own. It is also required that the two libraries named Raylib and Win32 be linked while compiling the project. The language being used for writing this code is C++.

3: SYSTEM ARCHITECTURE & DESIGN

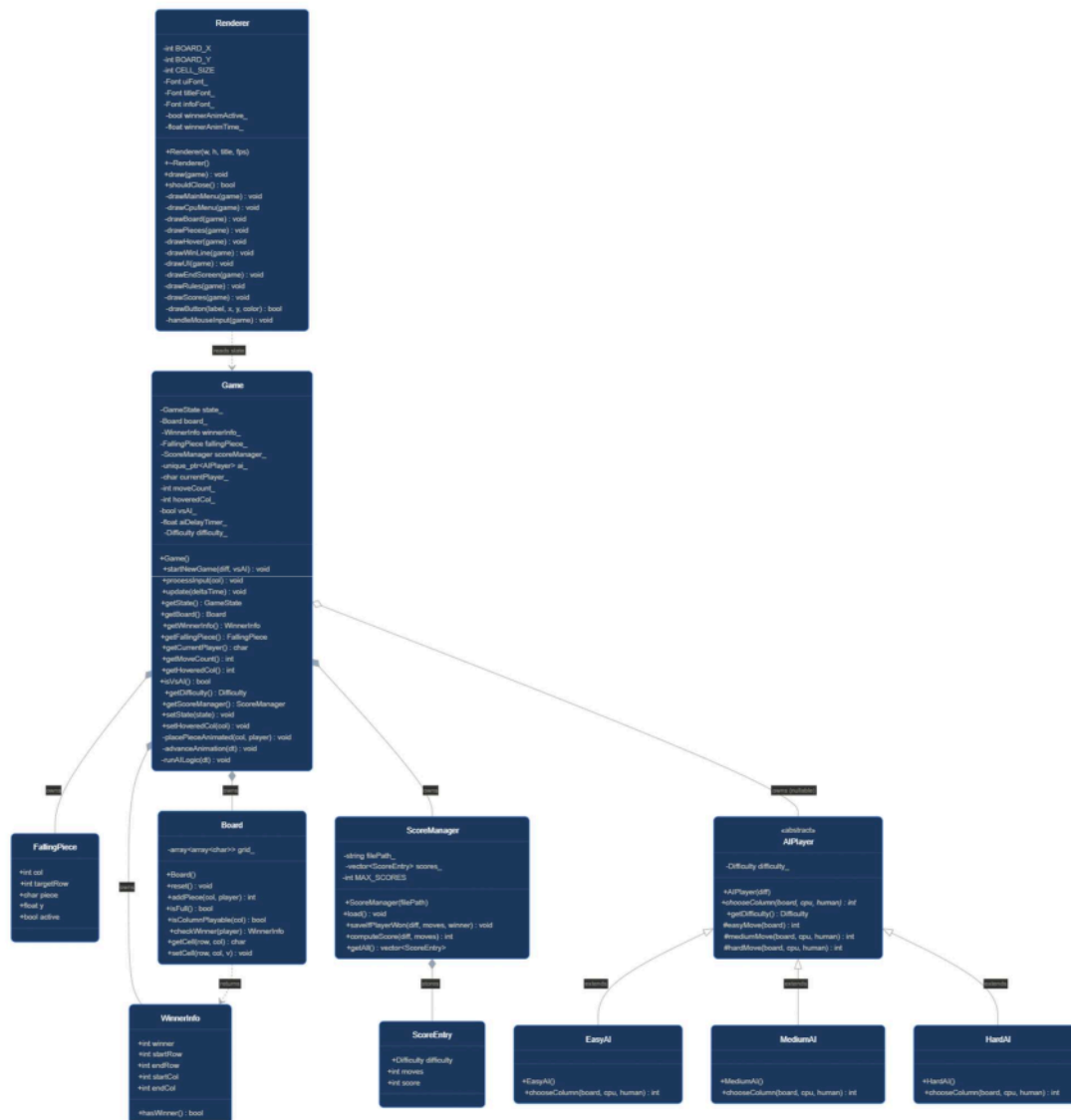
There are seven different architectural styles and two simple data structures and each one of these styles and data structures have their own tasks. Game class is responsible for all dynamic and non-dynamic entities. Everything related to Raylib is managed by the Renderer class. The grid is a responsibility of the Board class. The operations performed by the computer player are implemented by the AIPlayer hierarchy. ScoreManager class is responsible for guarding all files.

The direction of the dependencies is very restrictive: the Game class is dependent upon the Board class, AIPlayer class and ScoreManager class. The AIPlayer class is dependent upon the Board class. Renderer class is dependent upon the Game class. Lower level class is never aware of anything related to the higher level class such as Game class, Board class, AIPlayer class or ScoreManager class.

Class Dependency Table:

Class	Depends On
Renderer	Game (read-only via const accessors)
Game	Board, AIPlayer, ScoreManager, FallingPiece, WinnerInfo
AIPlayer (abstract)	Board (const&, read-only)
EasyAI / MediumAI / HardAI	AIPlayer (inherits)
ScoreManager	AIPlayer (for Difficulty enum)
Board	WinnerInfo (returns from checkWinner)
WinnerInfo	None
FallingPiece	None

UML Diagram:



4: FILE STRUCTURE & SEPARATION OF CONCERNS

For your C++ projects, it is customary for everyone to separate the files in each class. You have the header file which has an extension .h. This tells you what the class is able to perform. You will also have the other file which has .cpp extension. This tells how the class performs its operations.

Your header file acts like a promise. Your header file shows the capabilities of the class. It doesn't show how the class operates. The other file is only compiled once.

However, there is one rule in the project. In this case, your Board.h, AIPlayer.h, ScoreManager.h and Game.h will not include the raylib.h library. It is included only by Renderer.h. As such, the part of the project that does most of the logical operations will have no problem being compiled without raylib library. C++ project is easy to comprehend because of the following reasons. C++ projects such as yours are divided into classes. Board.h, AIPlayer.h, ScoreManager.h and Game.h are not dependent on raylib.h library.

File	Responsibility
WinnerInfo.h / .cpp	Pure data: winner flag and winning cell coordinates
FallingPiece.h	Pure data: animation state for the falling piece
Board.h / .cpp	The 6x7 grid, piece placement, and win detection
AIPlayer.h / .cpp	Abstract AI base class and three concrete difficulty implementations
ScoreManager.h / .cpp	File I/O for score persistence and score calculation
Game.h / .cpp	Game state coordinator – owns all logic objects
Renderer.h / .cpp	Everything Raylib – drawing, input, animation

main.cpp	Entry point – constructs Game and Renderer, runs the loop
----------	---

5: FUNDAMENTAL OOP CONCEPTS

- **Encapsulation**

The principle of Encapsulation depends on the idea that data and the processes that work with them can be bundled into a single component and that no other component can access the data contained within it.

The board in the C program was an exposed datum that could be accessed by any other program.

On the other hand, the grid in the C++ program is encapsulated in the Board class and, hence, not accessible to anything else.

```
class Board {
public:
    Board();

    void reset();
    int addPiece(int col, char player); // returns row placed, -1 if invalid
    bool isFull() const;
    bool isColumnPlayable(int col) const;

    WinnerInfo checkWinner(char player) const;

    char getCell(int row, int col) const; //getter for a specific cell
    void setCell(int row, int col, char value); //setter for a specific cell

private:
    std::array<std::array<char, COLS>, ROWS> grid_;
};
```

- **Abstraction**

Abstraction presents a simplified interface while hiding implementation complexity. The Game::update(float deltaTime) method is a perfect example:

```
void Game::update(float deltaTime) {
    advanceAnimation(deltaTime);
    if (vsAI_ && state_ == GameState::CpuGame) {
        runAILogic(deltaTime);
    }
}
```



```
}  
}
```

- **Inheritance**

Inheritance allows a child class to derive the interface of a parent class and override it. The AI system demonstrates this:

```
class AIPlayer {  
public:  
    explicit AIPlayer(Difficulty difficulty);  
    virtual ~AIPlayer() = default;  
  
    virtual int chooseColumn(const Board& board, char cpuPiece, char humanPiece) const = 0;  
  
    Difficulty getDifficulty() const;  
  
private:  
    Difficulty difficulty_;  
  
protected:  
    static int easyMove (const Board& board);  
    static int mediumMove(const Board& board, char cpu, char human);  
    static int hardMove (const Board& board, char cpu, char human);  
};  
  
class EasyAI final : public AIPlayer {  
public:  
    EasyAI();  
    int chooseColumn(const Board& board, char cpuPiece, char humanPiece) const override;  
};  
  
class MediumAI final : public AIPlayer {  
public:  
    MediumAI();  
    int chooseColumn(const Board& board, char cpuPiece, char humanPiece) const override;  
};
```

- **Polymorphism**

Polymorphism allows a single interface to represent different underlying types. Game holds the AI as a `unique_ptr<AIPlayer>`:


```
// Game calls the same line regardless of difficulty
int aiCol = ai_->chooseColumn(board_, 'O', 'X');
if (aiCol >= 0 && board_.isColumnPlayable(aiCol)) {
    placePieceAnimated(aiCol, 'O');
}
// At runtime, this dispatches to EasyAI, MediumAI, or HardAI
// based on whichever object was created by createAIPlayer()
aiDelayTimer_ = 0.0f;
}
```

6: MODERN C++ CONSTRUCTS: STRUCT, ENUM CLASS, WINNERINFO, FALLINGPIECE

Use of enum classes:

The original C enumeration did not have scopes, making the values such as EASY and MAIN_MENU global and implicitly convertible to integer types. enum class is capable of solving both issues:

```
enum class GameState {
    MainMenu, CpuMenu, PvpGame, CpuGame,
    ShowRules, ShowScores, EndScreen, ExitGame
};
```

WinnerInfo – C++ struct

```
struct WinnerInfo {
    int winner; // 0 = none, 1 = X, 2 = O
    int startRow, endRow;
    int startCol, endCol;

    bool hasWinner() const;
};
```

WinnerInfo is generated by Board::checkWinner(), and it is stored in Game, and then read by Renderer class to draw the win line and result text. The hasWinner() method co-locates the data.

FallingPiece – animation state

```
struct FallingPiece {
    int col = 0;
```

```

int    targetRow = 0;
char   piece     = ' ';
float  y         = 0.0f;
bool   active    = false;
};

```

The member initializers of the class make sure that default constructor of FallingPiece is safe and unactive, while ensuring that we don't have to separately initialize this class. Game::placePieceAnimated() sets active=true and the target; Game::advanceAnimation() advances y in each and when it lands; Renderer::drawPieces() reads the struct to draw the piece at its current y position.

7: GUI AND VISUALS

The Render Loop

```

while (!renderer.shouldClose() && game.getState() != GameState::ExitGame) {
    game.update(GetFrameTime());
    renderer.draw(game);
}

```

Inside Renderer::draw(), a switch on game.getState() connects to the correct private drawing method. The Renderer reads game state by accessing constant class methods, and writes back only through game.processInput() and game.setState() — by not directly modifying any Game member.

Piece Animation

When a piece is placed, Game::placePieceAnimated() calls the FallingPiece struct with the target row with an initial y position above the board. The piece is not committed to the grid yet. Each frame, Game::advanceAnimation() increments y by FALL_SPEED * deltaTime:

```

void Game::advanceAnimation(float deltaTime) {
    if (!fallingPiece_.active) return;

    fallingPiece_.y += FALL_SPEED * deltaTime;
    float targetY = BOARD_Y + fallingPiece_.targetRow * CELL_SIZE + CELL_SIZE / 2.0f;
    if (fallingPiece_.y >= targetY) {
        fallingPiece_.y = targetY;
        board_.setCell(fallingPiece_.targetRow, fallingPiece_.col, fallingPiece_.piece);
        moveCount_++;

        winnerInfo_ = board_.checkWinner(fallingPiece_.piece);
    }
}

```

```

        if (winnerInfo_.hasWinner()) {
            if (vsAI_ && fallingPiece_.piece == 'X') {
                scoreManager_.saveIfPlayerWon(difficulty_, moveCount_, winnerInfo_.winner);
            }
        } else {
            currentPlayer_ = (fallingPiece_.piece == 'X') ? 'O' : 'X';
        }
        fallingPiece_.active = false;
    }
}

```

Hover Effect

Renderer::drawHover() takes the raw mouse coordinates, determines which column is under the mouse cursor, and creates an overlay of translucent rectangles over that column creating a glowing effect. It reads board.getCell() to separate filled cells which are typically the red circles from empty cells which are the yellow circles, giving visual feedback on where the dedicated circle will drop.

Win Line Animation

The winning line is extended outwards from the centre point of the winning combination by performing lerp. The Renderer maintains its own animation state by using functions such as winnerAnimActive_ and winnerAnimTime:

```

float startXF = static_cast<float>(startX);
float startYF = static_cast<float>(startY);
float endXF = static_cast<float>(endX);
float endYF = static_cast<float>(endY);
float midX = (startXF + endXF) * 0.5f;
float midY = (startYF + endYF) * 0.5f;
float curStartX = midX + (startXF - midX) * t;
float curStartY = midY + (startYF - midY) * t;
float curEndX = midX + (endXF - midX) * t;
float curEndY = midY + (endYF - midY) * t;

```

8: DATA PERSISTENCE

Score Calculation

```

int ScoreManager::computeScore(Difficulty diff, int moves) const{
    int base, penalty, score;
    if (diff == Difficulty::Easy){base = 1000; penalty = 10;}
    else if (diff == Difficulty::Medium){base = 1200; penalty = 12;}
    else{base = 1500; penalty = 15;}
}

```

```
score = base - (moves * penalty);
if (score < 0) score = 0;
return score;
}
```

As the difficulty level of the question increases, the base value increases, since defeating the better player constitutes a significant victory. The deduction per move motivates players to carry out the game effectively. No negative scores are possible.

File loading with std::ifstream

```
void ScoreManager::load() {
    scores_.clear();
    ifstream file(filePath_);
    if (file.is_open()) {
        ScoreEntry entry;
        int diffValue = 0;
        while (file >> diffValue >> entry.moves >> entry.score) {
            if (diffValue < static_cast<int>(Difficulty::Easy) || diffValue >
static_cast<int>(Difficulty::Hard)) {
                continue;
            }
            entry.difficulty = static_cast<Difficulty>(diffValue);
            scores_.push_back(entry);
            if (scores_.size() >= MAX_SCORES) break;
        }
        file.close();
    }
}
```

File Writing with std::ofstream

```
void ScoreManager::saveIfPlayerWon(Difficulty diff, int moves, int winner){
    if (winner == 1){ // player won
        ScoreEntry newEntry{diff, moves, computeScore(diff, moves)};
        scores_.insert(scores_.begin(), newEntry);
        if (scores_.size() > MAX_SCORES) {
            scores_.pop_back();
        }
        ofstream file(filePath_, ios::trunc);
        if (file.is_open()) {
            for (const auto& entry : scores_) {
                file << static_cast<int>(entry.difficulty) << ' '
                    << entry.moves << ' '

```

```
        << entry.score << '\n';  
    }  
    file.close();  
}  
}
```

9: CHALLENGES & CONCLUSION

Technical Challenges

- **Managed State Migration:** Here, there were eight attributes that required proper management by an appropriate approach of ownership. The attributes were first located, and then their ownership was assigned to a respective class; thereafter only we proceeded to write a new code.
- **Integration of Raylib and Object-Oriented Programming:** Raylib includes a series of functions coded in C programming language. Since it had to be used in C++, the functions were put in the class carefully considering which functions were to go and which were not to be added to the class. All of Raylib types are defined in the files `Renderer.h` and `Renderer.cpp`.
- **Look-Ahead AI without Board amendments:** With a previous look-ahead implementation in C, it was necessary to mutate board elements and revert them to their initial state; however, it caused some glitches. Now our implementation duplicates the board and performs operations on the duplicated elements leaving the original board untouched.
- **Game vs. Renderer Animation Separation:** The position attribute plays a vital role in the gameplay and cannot be moved out of Game, but on the other hand, animated objects must be rendered by Renderer.

Conclusion

As we have switched from C to object-oriented C++, it is considered as a good thing for the source code. The code is now easy to read and modify. The code no longer uses any global variables. Each module within the code has its own programmer responsible for it. The game logic is decoupled from the Raylib library. This makes it possible to include the graphics library without affecting the Board, AIPlayer, Game, or ScoreManager codes.

Elements that may be integrated into the game include the combination of the minimax algorithm together with alpha-beta pruning in order to create an extremely hard level for winning. An online multiplayer function needs to be added to facilitate connection between players through the Internet. Users will have the freedom to choose the size of the playing board. Player statistics need to be recorded for a number of games. It is important to have a dynamic background for the menu screen.

Moazzamuddin Report

ORIGINALITY REPORT

0%

SIMILARITY INDEX

0%

INTERNET SOURCES

0%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

piembstech.com

Internet Source

<1%

Exclude quotes Off

Exclude bibliography Off

Exclude matches Off